

Smart QR Generator & Manager

Technical & Interview Preparation Guide

A full-stack QR platform: static & dynamic QR codes, customization, logo embedding, PNG/SVG/PDF export, dynamic redirects, privacy-conscious scan analytics, a documented REST API, and Docker.

FastAPI · SQLAlchemy · JWT · segno · Pillow · reportlab · OpenCV · vanilla JS · Chart.js · Docker · pytest

This guide teaches the actual implementation so you can explain and defend every part in an interview. It contains no invented metrics, users, or deployment claims — only what the code really does.

Contents

1. Three ways to explain the project (30s / 2min / full)
2. Architecture & request flows
3. Tech stack — and why each choice
4. QR code concepts
5. Static vs dynamic QR (the core idea)
6. Backend & FastAPI concepts
7. Database concepts
8. Security concepts
9. Analytics & privacy concepts
10. Code walkthrough (the files to know)
11. Interview questions & answers (by category)
12. Limitations & future improvements
13. CV bullet points (3 versions)
14. How to demo it live

1. Three ways to explain the project

30-second version (memorise this)

Elevator pitch

“It’s a full-stack web app for creating and managing QR codes. You can generate codes for links, Wi-Fi, contacts and more, customise them with colours and a logo, and export them as PNG, SVG or PDF. The interesting part is **dynamic QR codes**: instead of encoding the URL directly, the code encodes a short redirect link my server controls, so you can change where a printed code points *without reprinting it* — and I record privacy-conscious scan analytics. It’s built with FastAPI, SQLAlchemy, JWT auth, and Docker, with a REST API and a test suite.”

2-minute version

Start with the problem, then the design, then what you’re proud of:

- **Problem:** a normal QR code has its destination baked in. If you print it on a poster and the link changes, the code is dead. This app solves that with static and dynamic modes.
- **Static** codes encode the content directly (a URL, Wi-Fi credentials, a contact card). **Dynamic** codes encode a short redirect URL like `/r/aB3xK9p`; the server looks up the current destination, logs the scan, and redirects. The printed image never changes.
- **Backend:** FastAPI with a clean service layer. JWT authentication, bcrypt password hashing, and server-side ownership checks so users only touch their own codes.
- **QR pipeline:** I use the segno library for true-vector SVG and PDF, Pillow for logo embedding, and I *verify* every generated code by decoding it back with OpenCV and comparing to the expected payload.
- **Analytics:** for dynamic codes I record device, browser, OS and a coarse region — but I never store raw IP addresses, only a salted hash.
- **Engineering:** 56 passing tests including decode tests, a documented REST API with OpenAPI docs, Docker, and honest documentation of limitations.

Full technical version

Walk the end-to-end flow. “A request comes into FastAPI. For a protected endpoint, a dependency decodes the JWT into a user or returns 401. The request body is validated by a Pydantic schema. The route calls a service function — for example `qr_service` allocates a random `short_id` and builds the encoded text; for a dynamic code that text is my own `/r/<short_id>` endpoint, not the final URL. It’s saved through SQLAlchemy and returned as JSON.”

“When someone scans a dynamic code, they hit `GET /r/<short_id>` with no authentication. The redirect service resolves the code, checks it’s active, records a privacy-safe scan event, re-validates that the stored destination is still an http/https URL, and returns a 302 to the current destination.”

2. Architecture & request flows

Layers

```
Browser (frontend: HTML/CSS + vanilla JS, Chart.js)
|   fetch() + JWT in Authorization header
v
FastAPI app (app/main.py)
|
API routers (app/api/routes/) auth | qr | analytics | redirect
|   + dependencies: get_current_user, get_owned_project
v
Service layer (app/services/)
  payloads | qr_service | export_service | redirect_service
  analytics_service | auth_service
v
SQLAlchemy ORM models (app/db/models.py)
  User 1--* QRProject 1--* ScanEvent
v
SQLite (dev) / PostgreSQL (prod)
```

Thin routes, fat services. Routes handle only HTTP concerns; the real logic lives in testable service functions. There are deliberately no extra abstractions (no repository layer, no message queue) because the project doesn't need them.

Flow A — authenticated API request (create a QR)

```
POST /api/qr (Authorization: Bearer <token>)
-> get_current_user      decode JWT -> User, else 401
-> Pydantic QRCreate     validate body, else 422
-> qr_service            allocate random short_id; build content
                        (dynamic => encode BASE_URL/r/<short_id>)
-> DB commit (SQLAlchemy)
-> 201 Created + QRout JSON
```

Flow B — public scan of a dynamic QR (no login)

```
GET /r/aB3xK9p (a phone camera opened this URL)
-> resolve_project(short_id) 404 if missing, 410 if inactive
-> record_scan()             device/browser/OS + hashed IP
-> destination_for(project)  re-validate http/https
-> 302 Location: <current destination>
```

3. Tech stack — and why each choice

Choice	Why this, in one line
FastAPI	Async-capable, automatic OpenAPI docs, Pydantic validation, clean dependency injection.
SQLAlchemy ORM	One set of models runs on SQLite (dev) and PostgreSQL (prod); safe parameterised queries.
JWT (PyJWT)	Stateless auth: the signed token identifies the user, so no session table is needed.
bcrypt (passlib)	Deliberately slow, salted password hashing — the standard for storing passwords.
segno	Produces TRUE vector SVG and a PDF-ready module matrix, unlike the raster-first 'qrcode' lib.
Pillow	Image processing for the logo-embedding pipeline and raster PNG output.
reportlab	Draws the QR as vector rectangles into a printable, labelled PDF page.
OpenCV (headless)	Decodes generated QRs to verify they scan; self-contained, no system zbar dependency.
Vanilla JS + Chart.js	No build step, easy to understand and defend; charts vendored locally for offline use.
slowapi	Simple IP-based rate limiting on login, registration, preview, and redirect.
Docker	Reproducible container; compose file with an optional PostgreSQL profile.
pytest	56 tests including QR decode tests and authorization checks.

Interview line about choices

“I added technologies only when they solved a real problem. I chose segno over the more common qrcode library specifically because I needed true-vector SVG and PDF output; I chose OpenCV for decode-verification because it has no system dependency, which kept my Docker image simple.”

4. QR code concepts

Anatomy

- **Finder patterns:** the three big corner squares the scanner uses to locate and orient the code.
- **Timing & alignment patterns:** help the scanner map the grid of modules.
- **Data + error-correction codewords:** the payload plus redundancy.
- **Quiet zone:** a blank margin of at least 4 modules; without it, scanners often fail against a busy background.

Version and capacity

The **version** (1–40) sets the grid size; higher versions hold more data. The library picks the smallest version that fits your payload at the chosen error level. More data or higher error correction $\Rightarrow$ a denser code.

Error correction (Reed–Solomon)

Four levels — L $\approx$ 7%, M $\approx$ 15%, Q $\approx$ 25%, H $\approx$ 30% recoverable. Higher levels let the scanner rebuild damaged or covered modules, at the cost of a denser code. **A logo forces level H** so the covered modules can be recovered.

Why logos can break QR codes

A logo covers data modules. Error correction can rebuild covered modules only up to its limit, so a logo that's too big makes the code unrecoverable. My pipeline caps the logo at $\sim$ 22% of the width, paints a clean background patch behind it, and forces error level H — and a test decodes the result to prove it still scans.

5. Static vs dynamic QR — the core idea

Static

Encodes the content directly (the URL/Wi-Fi/contact is literally in the image). Works forever with no server, but changing the destination means reprinting.

Dynamic

Encodes a redirect URL I control — `BASE_URL/r/<short_id>`. The real destination lives in the database. On scan, the server looks it up, records the event, and 302-redirects. Because the image only contains the short link, **you can change the destination without reprinting the code.**

The guarantee, and how it's tested

Updating a dynamic code changes only `destination_url`, never `short_id`. The test `test_changing_destination_keeps_short_id_and_image` asserts the `short_id` is unchanged after an update and the redirect points to the new URL. There is no API to change a code's mode or `short_id`.

Edge cases handled

- Unknown `short_id` $\rightarrow$ 404.
- Deactivated/deleted code $\rightarrow$ 410 Gone (it stops redirecting even though the printed image still exists).
- No/invalid destination $\rightarrow$ 409.
- Dynamic mode is URL-only; a dynamic request for wifi/vcard/etc. is rejected with 422.

6. Backend & FastAPI concepts

- **REST:** resources (QR projects) manipulated with HTTP verbs — GET (read), POST (create), PUT (update), DELETE (remove) — and meaningful status codes.
- **Pydantic schemas:** every request body is a typed model; invalid input is rejected with 422 before any handler runs. Responses use `response_model` so only documented fields are returned.
- **Dependency injection:** FastAPI's `Depends` supplies the DB session, the current user (from the JWT), and the owned-project lookup. Shared logic is declared once and reused.
- **Auth dependency:** `get_current_user` decodes the Bearer token → User or 401. `get_owned_project` adds the ownership check.
- **Automatic docs:** FastAPI generates OpenAPI from the routes/schemas; Swagger UI is served at `/docs` (vendored locally so it works offline).
- **Middleware & handlers:** a rate-limit middleware, plus exception handlers that turn rate-limit hits into 429 and any unhandled error into a generic 500 (never a stack trace).

7. Database concepts

- **Schema:** three tables — `users`, `qr_projects`, `scan_events`.
- **Relationships:** User 1—* QRProject 1—* ScanEvent, with cascade delete so removing a project removes its scans.
- **Foreign keys:** `qr_projects.owner_id` → `users`, `scan_events.qr_id` → `qr_projects`.
- **Indexes** on the columns actually queried: `email` (login), `short_id` (every redirect), `owner_id` (listing), and `scan_events(qr_id, timestamp)` for analytics.
- **JSON columns** for content and style — different QR types need different fields, so a JSON blob beats a wide, mostly-NULL table.
- **Transactions:** writes go through a session and `commit()`; the request-scoped session is always closed via a generator dependency.
- **Portability:** the same ORM models run on SQLite and PostgreSQL; time-series aggregation is done in Python to avoid vendor-specific SQL date functions.

8. Security concepts

- **Password hashing:** `bcrypt` (salted, slow) via `passlib`; only the hash is stored. A test asserts the DB value is a `bcrypt` hash, not the plaintext.
- **Authentication:** stateless JWT signed with the app secret; the signature proves the request is from a logged-in user.
- **Authorization:** server-side ownership check returns 404 for another user's id — blocking the IDOR attack and hiding which ids exist.
- **ID enumeration:** public URLs use a random 7-char `short_id` (~1.5e12 combos), never the sequential integer id.
- **URL validation:** redirect destinations must be `http/https`; `javascript:`, `data:`, `file:`, etc. are rejected — and re-validated on every scan.
- **File uploads:** logos are size/format/dimension-checked and fully re-encoded, so original bytes are never served back.
- **Secrets:** read from `environment/.env` (git-ignored); `.env.example` documents them; logs never contain passwords/tokens/secrets.
- **Rate limiting:** IP-based limits on login, registration, preview, and redirect.

Be honest about what it does NOT do

No email verification, password reset, 2FA, or account lockout. The JWT sits in localStorage (an XSS bug could read it; httpOnly cookies + CSRF would be stronger). I block dangerous URL *schemes* but do not scan destinations for phishing/malware. Saying this out loud shows judgement — interviewers value it more than overclaiming.

9. Analytics & privacy concepts

- **Event logging:** one ScanEvent row per dynamic scan — timestamp, device category, browser/OS family, optional coarse country.
- **Privacy:** no raw IP is stored — only `sha256(secret + ip)[:16]`, which can't be reversed; a test verifies this.
- **Device categorisation:** the User-Agent header is parsed into mobile/tablet/desktop/bot.
- **Aggregation:** totals, a zero-filled 14-day time series, and device/browser/country breakdowns — computed in Python for DB portability.
- **Honesty:** demo scans from the seed script are flagged `is_demo` and the UI shows a 'synthetic demo scans' banner; real numbers reflect real scans only.
- **Known noise:** bots and repeat scans can inflate counts; location is approximate; UA detection is imperfect. I document these rather than hide them.

10. Code walkthrough — the files to know

If you understand these ten files, you understand the project. For each: what it does, the key functions, and the design decision worth mentioning.

app/services/payloads.py

Purpose: Turns a content dict into the exact text encoded into a QR, and validates it.

Key parts: `build_payload()` (per-type formats: url/text/wifi/vcard/email/phone), `validate_destination_url()` (http/https only).

Design decision: Validation lives in one well-tested place instead of six schemas. URL validation is security-critical because dynamic redirects send a browser to this value.

Watch out for: Reserved characters in Wi-Fi/vCard must be escaped or a password with a ';' breaks the payload — handled by the escape helpers.

app/services/qr_service.py

Purpose: Core generation: text → segno symbol → PNG, plus the logo pipeline, warnings, and decode-verify.

Key parts: `encoded_text_for()` (static vs dynamic branch), `render_png()`, `_embed_logo()`, `scannability_warnings()`, `decode_png()/verify_png()`, `generate_short_id()`.

Design decision: A logo forces error level H and is capped at ~22% width so the code still decodes; generation is verified by decoding it back with OpenCV.

app/services/export_service.py

Purpose: Produces the three download formats.

Key parts: `to_png()` (raster + logo), `to_svg()` (true vector), `to_pdf()` (draws the module matrix as vector rectangles + title/label).

Design decision: SVG/PDF are real vector output, not a raster wrapped in a container — a genuine, testable difference.

app/services/redirect_service.py

Purpose: Resolves a dynamic `short_id` to its current destination, safely.

Key parts: `resolve_project()` (404/410), `destination_for()` (re-validate http/https).

Design decision: Re-validating the destination on every scan is defence in depth — even a bad DB value can never send a scanner to a javascript: URL.

app/services/analytics_service.py

Purpose: Records scans (privacy-safe) and aggregates them.

Key parts: `hash_ip()`, `classify_user_agent()`, `record_scan()`, `qr_analytics()`, `dashboard_overview()`.

Design decision: Only a salted IP hash and coarse fields are stored; time-series is bucketed in Python to stay database-agnostic.

app/api/dependencies.py

Purpose: Reusable auth/ownership dependencies.

Key parts: `get_current_user()` (JWT → User or 401), `get_owned_project()` (loads a project AND checks ownership).

Design decision: Ownership returns 404 (not 403) for someone else's code, hiding whether the id exists. This is the single choke-point for authorization.

app/api/routes/qr.py

Purpose: QR CRUD, preview, logo upload, image, download, verify.

Key parts: `create_qr()`, `update_qr()`, `preview()` (public), `download()`, `project_to_out()` (adds `scan_count/redirect_url`).

Design decision: Static vs dynamic is decided at creation and made immutable; update only exposes `name/destination/active/style`.

app/api/routes/redirect.py

Purpose: The public GET `/r/{short_id}` endpoint.

Key parts: `follow_redirect()`, `_client_ip()` (X-Forwarded-For aware).

Design decision: Analytics recording is wrapped in try/except so a logging failure never blocks the redirect; unauthenticated by design.

app/db/models.py

Purpose: The three SQLAlchemy models and their indexes/relationships.

Key parts: `User`, `QRProject` (`short_id`, `content/style` JSON), `ScanEvent` (`ip_hash`, `is_demo`).

Design decision: `short_id` is a random unique column, not the PK; JSON columns keep the schema simple across six content types.

app/main.py

Purpose: App wiring: routers, static frontend, offline Swagger docs, rate limiter, exception handlers, table creation on startup.

Key parts: `create_all_tables()` on startup, custom `/docs`, generic 500 handler.

Design decision: One process serves the API and the static frontend for a single-command run; docs assets are vendored so `/docs` works offline.

11. Interview questions & answers

Curated across the categories interviewers actually probe for a project like this. Every answer matches the real implementation. Where useful, a short verbal framing and a likely follow-up are included.

Beginner

Q. What does this project do?

It's a web app to create, customise, download and manage QR codes, with static and dynamic modes and scan analytics for the dynamic ones.

Q. What is a REST API?

An interface where resources (here, QR projects) are manipulated over HTTP using verbs — GET to read, POST to create, PUT to update, DELETE to remove — with status codes indicating the outcome.

Likely follow-up: Which HTTP status codes did you use? → 200/201/204/401/404/409/410/422/429/500.

Q. What is the difference between static and dynamic QR codes?

A static code encodes the content directly; a dynamic code encodes a short redirect URL my server controls, so the destination can change without reprinting the code.

Say it like this: Lead with this — it's the heart of the project.

Q. Why FastAPI and not Flask/Django?

FastAPI gives typed request validation via Pydantic, automatic OpenAPI docs, and clean dependency injection out of the box, which fit a REST API well.

Q. Where are passwords stored?

Only as bcrypt hashes, never plaintext. Login re-hashes the candidate password with the stored salt and compares.

Q. What database did you use?

SQLite in development for zero setup, with the same SQLAlchemy models running on PostgreSQL in production by changing one connection string.

Q. How does a user log in?

They POST email+password; the server verifies the bcrypt hash and returns a signed JWT that the browser sends on later requests as a Bearer token.

Q. What formats can a QR be downloaded in?

PNG (raster), SVG (true vector), and a printable PDF page with the QR drawn as vector rectangles plus a title and label.

Q. What is a quiet zone?

The blank margin around a QR (at least 4 modules). Without it, scanners often fail to detect the code against a busy background.

Q. What happens when you scan a dynamic code?

The phone opens my /r/<short_id> URL; the server looks up the current destination, records the scan, and 302-redirects the browser there.

Q. How do you run the project?

Create a virtualenv, install requirements, copy .env.example to .env with a real secret, optionally seed demo data, then run `uvicorn app.main:app`.

Q. What is the frontend built with?

Plain HTML/CSS and vanilla JavaScript with Chart.js for charts — no framework or build step, so it's easy to read and serve.

Intermediate

Q. How is authorization enforced?

A dependency loads the project by id and checks `owner_id` equals the current user; if not, it returns 404. This runs on the server for every protected route.

Likely follow-up: Why 404 not 403? → it hides whether the id exists, preventing enumeration.

Q. How do you validate request bodies?

Each endpoint declares a Pydantic schema; FastAPI rejects invalid input with a 422 before the handler runs. Responses use `response_model` so only documented fields go out.

Q. Why store content and style as JSON columns?

Different QR types need different fields; a JSON blob avoids a wide table full of NULLs. I never need to query inside the payload, so the trade-off is fine.

Q. How do you keep the code testable?

Business logic lives in service functions that take a DB session and arguments, so I test them without HTTP. Routes are thin wrappers.

Q. How does the live preview work?

The generator POSTs the current options to a public `/api/qr/preview` endpoint that renders a PNG without saving, and returns readability warnings in a response header.

Q. What's in a JWT here and how is it verified?

A payload with the user id (sub), issued-at, and expiry, signed with HMAC-SHA256. The server decodes and verifies the signature with the secret; failure means 401.

Q. How do you handle errors globally?

Exception handlers turn rate-limit hits into 429 and any unhandled exception into a generic 500 with a logged detail, so stack traces never reach the client.

Q. Why is the redirect endpoint unauthenticated?

Anyone with a printed code must be able to scan it. The scan is public; the analytics it produces are private to the owner.

Say it like this: Good line: 'the redirect is public, the analytics are private.'

Q. How do you prevent a huge logo from breaking the code?

I cap the logo at ~22% of the width, paint a background patch behind it, and force error-correction level H — then a test decodes the result to confirm it scans.

Q. How is the database session managed per request?

A generator dependency yields a session and closes it in a finally block, so it's always cleaned up even if the handler raises.

Q. What does duplicate do?

It clones a project into a new one with a fresh `short_id` and no scan history, so you can reuse a design without touching the original's analytics.

Q. How do you show that a QR really scans?

A verify endpoint renders the code, decodes it with OpenCV, and returns whether the decoded text equals the expected payload — the UI shows a badge based on that.

Advanced

Q. Walk me through the dynamic redirect end to end.

GET `/r/<short_id>` → resolve the project (404 if missing, 410 if inactive) → record a privacy-safe scan → re-validate the stored destination is http/https → 302 to it. Analytics recording is wrapped so it can't block the redirect.

Q. Why re-validate the destination on every scan instead of only at save?

Defence in depth. Even if a bad value somehow reached the database, the scan path still refuses to send a browser to a javascript:/data: URL.

Q. How would this scale to millions of scans?

Move time-series aggregation into SQL with `date_trunc + GROUP BY` and proper indexes, add a read replica or a rollup table, and use PostgreSQL. The Python aggregation is deliberately simple for the current scale.

Likely follow-up: What breaks first? → the per-request Python aggregation over all events.

Q. Why segno instead of the qrcode library?

segno produces true vector SVG and exposes the module matrix I draw into a vector PDF; qrcode is raster-first via Pillow. I needed genuine vector output.

Q. How do you keep the app database-agnostic?

I use the SQLAlchemy ORM (no raw SQL) and do date bucketing in Python instead of vendor-specific SQL date functions, so the same code runs on SQLite and PostgreSQL.

Q. What's the trade-off of stateless JWTs?

No session table and easy horizontal scaling, but a token can't be revoked before it expires. I mitigate with a short lifetime; refresh tokens/blocklist would be the next step.

Q. How is the short_id generated and collisions handled?

7 random characters from a 55-symbol alphabet via the secrets module; the column is UNIQUE and the creating code retries a few times on the rare clash.

Q. How would you add QR expiry or scheduled redirects?

Add `valid_from/valid_until` (or a schedule) to the project and check them in `resolve_project`, returning 410 outside the window. It's a small, localised change because redirect logic is centralised.

Q. Where could this code have a bug, and how do you guard it?

The Wi-Fi/vCard payloads must escape reserved characters, colour contrast can silently hurt scanning, and analytics could block a redirect. I guard each with escaping, warnings, and a try/except around scan recording — plus decode tests.

Q. Why compute analytics in Python rather than SQL?

Portability and simplicity at this scale — SQLite and Postgres differ on date functions. I call out that SQL aggregation is the right move once data grows.

Project-specific

Q. Why did you build this?

To demonstrate a real, end-to-end backend: REST design, auth, a non-trivial domain (QR + redirects + analytics), image processing, and honest engineering — not a CRUD tutorial.

Q. What was the hardest part?

Getting logo embedding to stay scannable — balancing logo size, the background patch, and error correction — and proving it with a decode test rather than eyeballing it.

Q. What are you most proud of?

The dynamic redirect design and that generation is verified by decoding the output, so 'it scans' is tested, not assumed.

Q. How do you know your QR codes actually work?

Decode tests: I render each type and decode it back with OpenCV, asserting it equals the intended payload; there's also a live verify endpoint.

Q. How many tests and what do they cover?

56 tests covering auth, authorization, CRUD, static/dynamic generation, redirect behaviour, analytics, validation, exports, and decode round-trips.

Q. What would you change if you rebuilt it?

Add Alembic migrations from the start, move the token to httpOnly cookies with CSRF protection, and push analytics aggregation into SQL.

Q. Is any of the analytics data real?

No — the demo scans are synthetic, generated by a seed script, flagged `is_demo`, and labelled as such in the UI. Real deployments would show only real scans.

Say it like this: Volunteer this — honesty about demo data reads very well.

Q. How does a user change where a printed code points?

For a dynamic code they edit the destination in the app; it updates one DB row. The `short_id` and the printed image are untouched.

Q. What content types are supported and how?

URL, text, Wi-Fi, vCard, email, phone — each built with its standard payload format (e.g. `WIFI:....;`, `mailto:..`, `tel:..`, a VCARD block).

Q. How is a deleted code handled for someone who already printed it?

It returns 410 Gone and stops redirecting. Deleting also cascades to its scan events.

Q. What's the project structure?

`app/` split into `core`, `db`, `schemas`, `services`, `api`; a static frontend/; `tests/`; `scripts/` for seeding; `docs/`. Logic is in `services`, not in `main.py`.

Q. Did you deploy it?

Not to a public host as part of building it, so I don't claim a live URL or uptime. It runs locally and via Docker; the deployment steps are documented.

Q. How big is the codebase and did AI write it?

I can explain every file and decision here — that's the point of this guide. The value is understanding it, and I can modify any part on the spot.

Q. What does the seed script do?

Creates a demo user, five representative projects, and ~360 synthetic scans spread over 14 days, all flagged as demo. It's re-runnable.

Q. How do you export a printable QR?

The PDF endpoint draws the QR as vector rectangles on an A4 page with the project name and destination label, using reportlab.

Q. Why vanilla JS instead of React?

The app's interactivity is modest, and a no-build vanilla frontend is easier to understand and defend; it also varies my portfolio, which already has a React project.

FastAPI / backend

Q. What is dependency injection in FastAPI?

Declaring a function parameter with Depends(...) so FastAPI resolves and supplies it — I use it for the DB session, the current user, and ownership checks.

Q. How does FastAPI generate docs?

It builds an OpenAPI schema from your route signatures and Pydantic models; Swagger UI at /docs renders it. I vendored the UI assets so it works offline.

Q. What is Pydantic doing here?

Parsing and validating request bodies into typed objects and serialising responses; validation errors become 422s automatically.

Q. How do you return a non-JSON response like an image?

I return a Response with the bytes and the right media_type (image/png, image/svg+xml, application/pdf) and a Content-Disposition header for downloads.

Q. Sync or async routes — which did you use and why?

Mostly synchronous handlers, since the DB driver and image work are blocking; FastAPI runs them in a threadpool. I'd switch to async with an async driver if I needed high concurrency.

Q. How is rate limiting implemented?

slowapi with an IP key function; decorators set per-endpoint limits and a handler returns 429 when exceeded.

Q. How do you upload files?

An UploadFile parameter (multipart), then I validate size/format/dimensions and re-encode the image before storing it as base64.

Q. What is response_model and why use it?

It declares the output schema so FastAPI filters and documents the response — the API can't accidentally leak internal fields like a password hash.

Q. How do you handle configuration?

pydantic-settings reads typed settings from environment variables / .env, cached once per process, and overridable in tests.

Q. How do you test a FastAPI app?

With TestClient over an in-memory SQLite DB injected via dependency_overrides, so tests are fast and isolated.

Q. What is middleware and where did you use it?

Code that wraps every request/response; I use the slowapi middleware for rate limiting and exception handlers for 429/500.

Q. How does the JWT reach the server on each request?

The browser stores it and sends Authorization: Bearer ; an HTTPBearer security scheme extracts it and my dependency decodes it.

Database

Q. Describe your schema.

users, qr_projects, scan_events. Users own projects; projects own scan events; both relationships cascade on delete.

Q. What indexes did you add and why?

email (login lookup), short_id (every redirect), owner_id (listing a user's codes), and scan_events(qr_id, timestamp) for analytics — the columns actually filtered/joined.

Likely follow-up: What's the cost of an index? → slower writes and more storage, worth it for hot read paths.

Q. Why not expose the integer primary key in URLs?

Sequential ids are enumerable; a random short_id can't be guessed, so people can't discover others' codes by counting up.

Q. How do foreign keys and cascades work here?

owner_id and qr_id are FKs; cascade delete removes a user's projects and a project's scans, so no orphaned rows linger.

Q. How do you migrate the schema?

Currently create_all on startup creates missing tables. For evolving schemas I'd add Alembic migrations — I list that as a known gap.

Q. SQLite vs PostgreSQL — when to switch?

SQLite is great for dev and low concurrency; I'd move to PostgreSQL for concurrent writes, real deployment, and heavier analytics.

Q. How do you avoid SQL injection?

I use the SQLAlchemy ORM with parameterised queries; user input never gets string-concatenated into SQL.

Q. What's a transaction and where does one occur?

A unit of work committed atomically; each create/update/delete does its work and commits through the request-scoped session.

Q. Why JSON columns rather than more tables?

The per-type fields vary; JSON keeps one simple table instead of a sparse wide one or a join per type. I trade away in-SQL querying of the payload, which I don't need.

Q. How would you query 'scans per day' efficiently at scale?

GROUP BY a date_trunc('day', timestamp) with an index on timestamp, instead of pulling rows and bucketing in Python as I do now.

Security

Q. How are passwords protected?

bcrypt hashing with a per-password salt; only the hash is stored and bcrypt's slowness resists brute force. A test asserts the stored value isn't the plaintext.

Q. What is IDOR and how do you prevent it?

Insecure Direct Object Reference — reaching another user's resource by editing an id. I prevent it with a server-side ownership check that returns 404 for someone else's code.

Say it like this: Name-drop 'IDOR' — it signals you know the vocabulary.

Q. Why return 404 instead of 403 for another user's code?

403 confirms the id exists; 404 hides that, so an attacker can't map which ids are real.

Q. How do you stop malicious redirect URLs?

Only http/https destinations are allowed; javascript:, data:, file:, vbscript:, ftp: are rejected, and the check runs again on every scan.

Likely follow-up: Can you fully stop phishing? → No; I block schemes and allow deactivation, but don't scan content — I say so.

Q. How are secrets managed?

Environment variables via .env (git-ignored), documented in .env.example; the JWT secret and DB URL are never hard-coded or logged.

Q. Is the file upload safe?

The logo is size/format/dimension-validated and fully re-encoded by Pillow, so crafted bytes aren't passed through.

Q. What about XSS/CSRF?

The API is Bearer-token based, so classic CSRF doesn't apply; but the token in localStorage is exposed to XSS. httpOnly cookies + CSRF tokens would be stronger — a documented trade-off.

Q. How does rate limiting help security?

It slows credential-stuffing on login/registration and abuse of the public redirect. It's per-process/in-memory here; a distributed limiter would be the next step.

Q. Do you leak stack traces?

No — a catch-all handler returns a generic 500 and logs the detail server-side.

Q. How do you keep JWTs from being tampered with?

They're signed with HMAC-SHA256; any change invalidates the signature and the request is rejected with 401.

Q. What sensitive data do you store about scanners?

None raw — only a salted, truncated IP hash plus coarse device/browser/OS and optional country. The raw IP is never persisted.

Q. What are the top risks you'd fix before real production?

Token revocation/refresh, email verification + password reset, moving the token to httpOnly cookies, and a distributed rate limiter.

QR codes

Q. What is error correction and which level do you use?

Reed–Solomon redundancy (L/M/Q/H, ~7–30%). Default M; a logo forces H so covered modules can be rebuilt.

Q. Why can a logo make a QR unscannable?

It covers data modules; beyond the error-correction limit the code can't be reconstructed. I cap logo size and raise correction to H.

Q. What is the quiet zone and why does it matter?

The blank border (≥ 4 modules) that lets scanners isolate the code; too small and detection fails.

Q. How is a Wi-Fi QR encoded?

As WIFI:T:WPA;S:ssid;P:password;;, with reserved characters escaped so a password containing ; or : doesn't break it.

Q. How is a contact QR encoded?

As a VCARD 3.0 block (N/FN/TEL/EMAIL/ORG/URL/ADR), with commas and semicolons escaped.

Q. What determines a QR's size/version?

The amount of data and the error-correction level; the library picks the smallest version that fits.

Q. How do you verify a generated code scans?

Decode it back with OpenCV and compare to the expected text — in tests for every type and via a verify endpoint.

Q. PNG vs SVG vs PDF for a QR — what's the difference?

PNG is raster (blurs when scaled); SVG is true vector (scales cleanly); the PDF draws vector rectangles onto a printable, labelled page.

Q. What makes a QR hard to scan besides logos?

Low contrast, inverted colours (light-on-dark), and too-small a quiet zone; I warn about each before saving.

Q. Can a dynamic QR's image change when the destination changes?

No — it encodes the short_id, which never changes, so the printed image is stable while the destination is editable.

Q. What library generates the codes and why?

segno, for true vector SVG and matrix access for the PDF; Pillow handles the logo composite and PNG.

Q. How many URLs can a short_id represent?

7 characters from a 55-symbol alphabet $\approx 1.5 \times 10^{12}$ combinations — effectively unguessable and non-colliding at this scale.

System design

Q. Sketch the high-level architecture.

A vanilla-JS frontend calls a FastAPI backend (routers → services → SQLAlchemy → SQLite/Postgres). A public `/r/` endpoint handles scans; everything else is JWT-protected.

Q. How would you make it highly available?

Run multiple stateless app instances behind a load balancer (JWT means no sticky sessions), move to PostgreSQL with a replica, and put the frontend on a CDN.

Q. Where's the bottleneck under heavy scanning?

The redirect path writes a row and the analytics endpoints aggregate in Python. I'd batch/queue scan writes and aggregate in SQL or a rollup table.

Q. How would you add caching, and would you?

Only if measured. Rendered images could be cached, but they're cheap to regenerate; a hot `short_id`→destination lookup is a better cache candidate (e.g. Redis).

Q. How do you separate frontend and backend for production?

Serve the static frontend from a CDN/nginx and run the API separately; the code only couples them through a couple of static routes.

Q. How would you support custom short domains?

Store a domain per project/account and build the encoded URL from it; the redirect resolver stays the same.

Q. How would you prevent redirect abuse at scale?

Scheme allow-listing (done), destination reputation checks, per-owner rate limits, and easy deactivation/abuse reporting.

Q. What would a queue add here?

Decoupling scan-writes from the redirect so a spike doesn't slow redirects — record to a queue, persist asynchronously.

Docker / DevOps

Q. What does your Dockerfile do?

Starts from `python:3.11-slim`, installs the two system libs OpenCV needs, installs Python deps in a cached layer, copies the app, and runs `uvicorn` as a non-root user.

Q. Why run as a non-root user in the container?

Least privilege — if the app is compromised, the attacker isn't root inside the container.

Q. What's in docker-compose?

The app service on SQLite by default, plus an optional PostgreSQL service behind a 'postgres' profile with a persistent volume.

Q. How do you keep secrets out of the image?

They come from `.env` at runtime (`env_file`), which is git-ignored and `.dockerignore'd`; nothing secret is baked into the image.

Q. Did you verify the image builds?

The compose config validates and the Dockerfile parses, but the registry was blocked where I assembled it, so I document that the full build should be run locally. I don't claim it as verified.

Say it like this: Honesty about an unverified build is better than a false claim.

Q. How would you deploy this?

Behind nginx/Caddy for TLS, uvicorn with multiple workers (or gunicorn), PostgreSQL, and env-based config; frontend on a CDN.

Q. How is configuration environment-specific?

One .env drives ENVIRONMENT, SECRET_KEY, BASE_URL, and DATABASE_URL; the same image runs in dev and prod with different values.

Q. Why does BASE_URL matter for Docker/deploy?

It's encoded into every dynamic QR as BASE_URL/r/; it must be the real public domain before you print dynamic codes.

Tricky follow-ups

Q. You said it's secure — prove it.

Passwords are bcrypt-hashed (tested), authorization is a server-side ownership check returning 404 (tested), redirect schemes are allow-listed (tested), and uploads are re-encoded. I also state what it doesn't do.

Q. What if two codes get the same short_id?

The column is UNIQUE and generation retries on a clash; at 1.5×10^{12} combinations it's astronomically unlikely.

Q. What if the analytics write fails during a redirect?

It's wrapped in try/except and logged; the redirect still happens because the visitor's experience matters more than a log row.

Q. Your analytics could be inflated by bots — so are they meaningful?

They're directional, not exact. I categorise obvious bots and am upfront that repeat scans and prefetchers add noise; de-duplication is future work.

Q. Isn't storing an IP hash still personal data?

It's pseudonymous and salted with a secret, not reversible to an address, and I store no raw IP — a deliberate privacy trade-off, documented in the privacy notice.

Q. Why not just use a URL shortener?

Dynamic QR is a shortener plus QR generation, ownership, analytics, and scan-time safety — and the QR image is the artefact, not the link.

Q. What breaks if the server goes down?

Dynamic codes stop redirecting (they depend on the server); static codes keep working because their content is self-contained.

Q. Can I change a static code's destination?

No — static content is baked into the image. That's exactly the limitation dynamic codes remove.

Q. Where would this fail a security review?

Token revocation, XSS exposure of the localStorage token, no email verification, and basic rate limiting. I know these and can prioritise fixes.

Q. If I gave you a day, what would you improve first?

Alembic migrations and moving analytics aggregation into SQL — they most affect correctness and scale; then httpOnly-cookie auth.

12. Limitations & future improvements

Stating these earns more trust than hiding them; full detail is in docs/limitations.md and docs/future-improvements.md.

Honest limitations

- No system guarantees a QR scans in every physical condition; decode-verify is a strong signal, not a promise.
- Analytics: approximate location, imperfect UA detection, bot/repeat-scan noise, Python-side aggregation.
- Security: no email verification/reset/2FA; JWT in localStorage; scheme allow-list but no phishing/malware scanning; basic rate limiting; no token revocation.
- Ops: SQLite default; no Alembic migrations yet; Docker build validated for config only; not deployed publicly (no uptime/perf numbers claimed).

Realistic next steps

- QR expiry, scheduled redirects, A/B destinations, campaign tags.
- SQL-side analytics aggregation; unique-visitor estimation; bot filtering.
- API keys + usage limits; webhooks; Redis-backed distributed rate limiting.
- Alembic migrations; httpOnly-cookie auth + CSRF; email verification/reset/2FA; CI running the tests.

13. CV bullet points (three truthful versions)

Pick the version matching the role. None invent metrics; each describes what the code really does.

Technical (full-stack)

- Built a full-stack QR management platform (FastAPI, SQLAlchemy, vanilla JS) supporting static and dynamic QR codes with a documented REST API and OpenAPI docs.
- Implemented dynamic QR redirects via unguessable short IDs, letting a printed code's destination change without reprinting, with server-side scan tracking.
- Added JWT authentication, bcrypt password hashing, and server-side ownership checks; wrote 56 pytest tests including QR decode-verification and authorization cases.
- Containerised the app with Docker and a docker-compose PostgreSQL profile; configuration via environment variables.

Backend-focused

- Designed a REST API in FastAPI with a clean service layer, Pydantic validation, dependency-injected auth, and consistent HTTP status/error handling.
- Modelled a normalized schema (users, QR projects, scan events) in SQLAlchemy with appropriate indexes and cascade deletes; SQLite in dev, PostgreSQL-ready.
- Enforced authorization server-side (IDOR-safe 404s) and validated redirect URLs to an http/https allow-list, re-checked on every scan.
- Built privacy-conscious analytics that store a salted IP hash and coarse device/browser data — never raw IP addresses.

Product-focused

- Shipped a QR generator/manager for links, Wi-Fi, contacts, email and phone, with colour/logo customisation and PNG, SVG and PDF export.
- Differentiated static vs dynamic codes and let users edit a live code's destination and view scan analytics through a clean, responsive dashboard.
- Added readability safeguards (contrast/quiet-zone warnings and decode-verification) so generated codes are actually scannable.
- Documented the system end-to-end (architecture, security, privacy, limitations) for maintainability.

14. How to demo it live

A tight five-minute demo that hits every strong point:

- 1. Open the landing page; type a URL in the live demo to show a real, rendered QR.
- 2. Log in with the demo account; show the dashboard — totals and the 14-day scan trend (note the scans are synthetic).
- 3. Go to Generate: switch content types (URL → Wi-Fi → Contact) and watch the live preview and warnings update.
- 4. Toggle Static vs Dynamic and explain the redirect idea in one sentence.
- 5. In My QR Codes, open a code to show the 'verified scannable' badge, then download a PDF.
- 6. Edit a dynamic code's destination; point out the `short_id` (and image) didn't change.
- 7. Open `/docs` to show the REST API, and run the test suite (`pytest -q`) to show 56 green.

Closing line for the interview

"Everything I've shown is tested and honest — the analytics are clearly-labelled demo data, I haven't claimed a deployment I didn't do, and I can walk you through any file or change any part right now."